

ENGINEERING JOURNAL

CALL.E — Problems Faced & How They Were Solved

A chronological record of every real problem hit while building CALL.E's knowledge-agent chatbot platform — latency, database load, session state, prompt grounding, concurrency, and production deployment — each with the actual root cause, the fix that was applied, and the measured result.

16

PROBLEMS DOCUMENTED

29.8s → 1.2s

/MESSAGE LATENCY, WORST TO FINAL

8 → 0

POSTGRES QUERIES PER MESSAGE

Contents

01	Gemini "thinking" model latency	GENERATION
02	Gemini 2.5 blocked on this API project	GENERATION
03	The greeting didn't need an LLM call	LATENCY
04	TTS audio generated nobody played	LATENCY
05	Eight database round-trips per message	DATABASE
06	A model 15-20× slower than it needed to be	GENERATION
07	Redis as the active-session source of truth	REDIS
08	The prompt let the model fill gaps with guesses	GROUNDING
09	Stricter prompt broke a truncation guard	GROUNDING
10	Would 20 concurrent users actually work?	LOAD TEST
11	One or two Gemini keys wasn't enough	RESILIENCE
12	Flask's dev server isn't a production server	DEPLOY
13	Gunicorn crashed under concurrent cold-start load	DEPLOY
14	The "clean" fix introduced a worse crash	DEPLOY
15	A codebase that had outgrown its own history	CLEANUP
16	Closing a tab left the conversation "active" forever	DATA INTEGRITY

Gemini "thinking" model latency

The very first live tests of the LLM layer were unusable — replies took anywhere from a few seconds to nearly a minute, with no obvious pattern.

PROBLEM

Raw calls to Gemini's default "thinking" model ranged from 2.79s to 46+ seconds, with high variance. One call failed outright after a 46-second wait, then hit a 429 rate limit on retry. The model spends a large, variable amount of hidden reasoning time per call before producing any visible output, and that reasoning occasionally leaked into the visible answer as garbled partial text.

FIX

Added `thinking_config=types.ThinkingConfig(thinking_budget=512)` to every generation call in `services/llm.py`, capping how many tokens the model can spend on hidden reasoning. This bounded both the latency variance and the reasoning-leak quality bug in one change.

Result: latency became bounded and predictable, though still slow on the "thinking" model — the real fix for raw speed came later in Chapter 06.

Gemini 2.5 turned out to be blocked entirely

When asked whether switching to Gemini 2.5 would help, the honest answer required actually testing it rather than assuming.

PROBLEM

`gemini-2.5-flash` returned a 404: *"This model is no longer available to new users."* Testing confirmed this applies to every 2.5 variant on this API project — `flash`, `flash-lite`, and `pro` all rejected identically, each pointing at a 3.x replacement.

FIX

No code fix needed — confirmed the constraint directly against the live API rather than guessing, and used Google's own error message to identify the correct replacement model family (3.x) to standardize on instead.

The greeting didn't need an LLM call at all

`/start` was taking ~20.8 seconds — as slow as a real answer — for a message that never actually varied in any way that mattered.

PROBLEM

The agent's opening line was generated by Gemini on every new conversation, using a template that asked for "a short, friendly opening message." But every field that message needed — agent name, role, organization name — was already sitting in the database. The LLM call was paying the full "thinking model" latency tax for a message with no real creative content.

FIX

Replaced `generate_greeting()` with a fixed f-string template built directly from `agent.name`, `agent.role`, and `company.name` — zero LLM calls.

`/start` latency: 20.8s → 2.3s

TTS audio was being generated that nobody played

Even after fixing the greeting, `/start` was still taking ~ 4.9 seconds. The remaining cost turned out to be entirely wasted work.

PROBLEM

`/start` was still calling Azure TTS to synthesize spoken audio for the greeting — a leftover from when the frontend had a voice-call UI. The rebuilt text-chat UI never read the `audio_url` field from the response at all. TTS synthesis alone measured ~ 1.4 seconds, paid on every single new chat for no benefit.

FIX

Removed the `generate_speech()` call from `/start` entirely, confirmed via `grep` that the frontend had zero references to the audio fields it used to consume.

`/start` latency: 4.9s → 2.3s

Eight database round-trips, on every single message

This is the problem that motivated everything in Chapter 07. Once Gemini itself got fast, the database quietly became the bottleneck — and it was worse than it looked.

PROBLEM

Direct SQL instrumentation (a SQLAlchemy `before_cursor_execute` event listener, run against a genuinely fresh request-scoped session — not a warmed-up one, which under-counts) showed **8 separate round-trips to Supabase's hosted Postgres** for a single `/message` call:

1. `SELECT conversations` — load the conversation row
2. `SELECT agents` — lazy-loaded relationship, not fetched with #1
3. `SELECT companies` — same, a second lazy-loaded relationship
4. `SELECT messages` — full history, for the prompt
5. `INSERT messages + commit` — save the user's message
6. `SELECT conversations` *again* — SQLAlchemy's default `expire_on_commit` silently re-fetches the row the moment its expired fields are touched after a commit
7. `SELECT count(*)` — tracked a message count that, by this point, nothing downstream even used
8. `INSERT messages + commit` — save the assistant's reply

Each round-trip cost 0.2–1s over the network to Supabase, not a local database — that's ~1.6-8s of pure database overhead stacked onto every single turn, on top of whatever the LLM itself cost.

FIX

This is documented as its own chapter (07) because the real fix wasn't tuning these eight queries — it was removing the database from the hot path entirely via Redis.

A model that was 15-20× slower than it needed to be

Investigating "can we use Gemini 2.5" (Chapter 02) surfaced a much bigger win sitting one config line away.

PROBLEM

The default model, `gemini-3.6-flash`, is a full "thinking" model — even capped at a 512-token thinking budget, live turns still measured 14-21 seconds. Listing every model actually available on the API project (rather than guessing at names) surfaced several lighter "lite" variants that had never been tried.

FIX

Benchmarked `gemini-3.1-flash-lite` and `gemini-3.5-flash-lite` head-to-head against the incumbent, using the real production prompt (not a toy one). Both lite models answered coherently and correctly grounded, in roughly a fifteenth of the time. Switched the default model.

Pure LLM call: 14.2-21.6s → ~1-1.3s

Redis as the active-session source of truth

The single largest architectural change of the whole project — directly answering Chapter 05.

PROBLEM

Even with a fast LLM, every message still paid for 8 Postgres round-trips (Chapter 05). Postgres was being treated as both the live working memory of an in-progress conversation *and* the permanent historical record — two very different access patterns forced through the same, network-latency-bound store.

FIX

Introduced Upstash Redis as a dedicated session cache (`services/session_store.py`), holding the complete state of one active conversation — agent snapshot, company snapshot, message history, metadata — under key `conversation:<id>` with a 24-hour TTL, refreshed on every write.

- `/start` reads Postgres once (conversation + agent + company), builds the session, writes it to Redis.
- `/message` touches *only* Redis — GET, run RAG + Gemini, append both turns, SET. Zero Postgres queries. If the session is missing, a clean error is returned rather than silently reconstructing state from Postgres.
- `/end` reads the Redis history, writes every message plus a generated summary to Postgres, and only deletes the Redis key after that commit succeeds — a failed Postgres write leaves the session intact rather than losing it.

Verified live: conversation isolation between two concurrent chats (one conversation's history never leaked into another's), and correct persistence of all 5 messages to Postgres in order after `/end`.

Postgres queries per message: 8 → 0

Warm `/message` latency: ~29.8s → ~1.2s

The prompt let the model fill gaps with guesses

Speed and cost were solved. The next problem was trust: would the agent ever invent a fact it shouldn't have?

PROBLEM

The original prompt asked the model to "prefer retrieved information" and "avoid inventing facts," but left real latitude — nothing stopped it from filling a gap in the knowledge base with a plausible-sounding general answer, e.g. inventing a helpline number or a procedure that was never actually in the source documents.

FIX

Rewrote the prompt around 15 explicit rules: answer only from retrieved knowledge / organization info / history; never fabricate URLs, phone numbers, IDs, dates, deadlines, eligibility, or amounts; explicitly refuse an unsupported personal-data question rather than inventing a plausible-sounding procedure; answer only the supported part of a multi-part question and say what can't be determined; absence of information is itself meaningful. Retrieved knowledge, organization info, history, and the question are each wrapped in their own XML-style tag so the model can't blur one input into another.

Verified live against the spec's own trap case — asked "What is my beneficiary ID?" with no such data in the knowledge base — and got back almost the exact expected refusal: *"I don't have access to your personal beneficiary ID, and the available information does not provide it."*

The stricter prompt broke an unrelated safety guard

Chapter 08's fix immediately exposed a bug in code that hadn't been touched at all — a good reminder that a prompt change can break things three layers away.

PROBLEM

A pre-existing guard, `_truncate_runaway_reply()`, chopped every reply at its first blank line — a reasonable heuristic under the old prompt, which explicitly demanded "one short, unbroken block of text." The new grounding prompt (Chapter 08) explicitly *invites* bullet points and multi-paragraph answers for multi-part questions, which legitimately contain blank lines. Real, correctly-grounded, fully-formed answers were being silently truncated mid-list.

FIX

Tightened the guard to only fire when a blank line is immediately followed by an actual fabricated speaker turn — `User:` or the agent's own name + colon, the exact pattern the conversation-history formatter itself uses — rather than any blank line at all. Verified with both a bullet-list answer (preserved intact) and a synthetic fake-dialogue continuation (correctly still truncated).

Would 20 concurrent users actually work?

Rather than guess, this was tested directly — and the first assumption going in turned out to be wrong.

PROBLEM

The expectation was that Flask's built-in dev server would serialize requests one at a time — a classic, well-known limitation. An isolated synthetic test (20 concurrent requests to a bare `time.sleep(2)` route) completed in ~2.03 seconds, not the ~40 seconds true serialization would produce. Reading Flask 3.1's own source confirmed why: `options.setdefault("threaded", True)` — a default that changed from older Flask versions this project's mental model was still based on.

FIX

Ran the real 20-user scenario (create → start → ask a real question) concurrently against the live pipeline. All 20 succeeded. But the server log revealed something the success rate alone hid completely.

METRIC	RESULT
Success rate	20/20 (100%)
/message latency under load	min 1.71s, median 2.84s, max 5.43s
DB pool exhaustion, Redis errors	None

One or two Gemini keys wasn't enough

The load test in Chapter 10 didn't just confirm the app scales — it proved a specific mitigation was load-bearing, not decorative.

PROBLEM

Buried in the server log from the 20-user test: `Gemini key index 0 rate-limited, trying next key` appeared 6 times, then the same for key index 1, another 6 times. A single free-tier key clearly cannot absorb a 20-person burst. Without automatic key rotation, an estimated 12 of those 20 concurrent requests — 60% — would have failed outright with a 429.

Later, after the user added 10 provisioned keys named `GEMINI_API_KEY_1` through `_10`, a second, quieter bug surfaced: the config loader only ever read a *bare* `GEMINI_API_KEY` plus `_2` through `_10` — key `_1` was being silently dropped every time.

FIX

Rebuilt the retry logic in `services/llm.py` into a genuinely strong mechanism: every key gets up to 3 attempts with backoff (1.5s, 3s) for *any* failure — not just 429s, also 5xx and raw network exceptions — before rotating to the next key, cycling through all configured keys (up to 30 total attempts across 10 keys) before finally raising. Fixed `config.py` to read `_1` through `_10`. Added a `threading.Lock()` around the shared "last-working-key" index, closing a race that existed when it was a bare global read/written from multiple concurrent request threads.

Flask's dev server isn't a production server

Everything so far had been tested against `python app.py` — fine for development, explicitly unsafe for real deployment.

PROBLEM

The dev server prints its own warning on every boot: *"This is a development server. Do not use it in a production deployment."* No process manager, no worker model suited to real concurrent traffic, no resilience to a crashed worker.

FIX

Added Gunicorn (`backend/gunicorn.conf.py`) — 2 worker processes $\times$ 4 threads each, using the `gthread` worker class specifically, since the default `sync` worker silently ignores `--threads` entirely. Set a 120-second request timeout, since Gemini replies can legitimately take several seconds. Added a `Procfile` so platforms like Heroku, Render, and Railway pick up the start command automatically. `app.py` itself needed zero changes — Gunicorn imports the exact same `app = create_app()` object the dev server already used.

Gunicorn crashed under concurrent cold-start load

The Gunicorn config alone looked complete on paper. Testing it under the same kind of concurrent burst as Chapter 10 immediately proved otherwise.

PROBLEM

Firing 8 simultaneous brand-new conversations at freshly-booted Gunicorn workers reliably crashed a worker with SIGSEGV. The server log showed `Loading weights` fire **six separate times** across the two worker processes before the crash — the smoking gun. The embedding-model singleton in `services/embeddings.py` had no lock: `if _model is None: _model = SentenceTransformer(...)`. When several threads inside the same `gthread` worker take their very first RAG-touching request at once, they all read `_model is None` together and each starts loading the PyTorch model concurrently — not a safe operation, and it segfaulted.

FIX

Added a `threading.Lock()` around the singleton's check-and-load, the standard double-checked locking pattern:

```
# services/embeddings.py
if _model is None or _model_name != model_name:
    with _model_lock:
        if _model is None or _model_name != model_name:
            _model = SentenceTransformer(model_name)
            _model_name = model_name
```

The lock only guards the one-time load; every later call finds `_model` already set and returns immediately, so it costs nothing on the hot path.

The "clean" fix introduced a worse crash

Not every attempted fix in this project worked — this one is included because backing out of a bad idea quickly is as much a real engineering decision as the idea itself.

PROBLEM

Before landing on the lock in Chapter 13, a different fix was tried first: `preload_app = True` plus an `on_starting` hook to warm the embedding model once in Gunicorn's master process before forking — elegant on paper, since every worker would then inherit an already-loaded model via copy-on-write memory, with zero changes to RAG pipeline code at all. It backfired immediately and worse: both workers crashed simultaneously on plain `/start` calls — before the embedding model was even touched — with `objc[...]: +[NSMutableString initialize] may have been in progress in another thread when fork() was called. Crashing instead. Preloading had also forked the master's already-open Supabase/httpx connection state into both children, which is unsafe to share across a fork.`

FIX

Reverted `preload_app` entirely rather than layering a second workaround (a `post_fork` hook disposing the DB engine) on top of an approach that had already proven fragile. Went back to the simpler, already-safe non-preloading config and applied the direct fix instead (Chapter 13's lock).

What's still open: even with the lock, an even deeper, distinct issue surfaced under the same stress test — PyTorch's BLAS/OpenMP thread pool initializing unsafely as the very first thing inside a freshly-forked worker process, a documented class of Apple Accelerate + `fork()` incompatibility. Standard mitigations (`OMP_NUM_THREADS=1`, `VECLIB_MAXIMUM_THREADS=1`, `OBJC_DISABLE_INITIALIZE_FORK_SAFETY=YES`) only partially resolved it on this development machine. This was not verified to reproduce on Linux — the actual deployment target, which uses a different, more fork-tolerant BLAS stack — so it's documented as a known local-only caveat in the architecture reference rather than silently shipped or silently ignored.

A codebase that had outgrown its own history

CALL.E started as a voice sales-calling agent and was repositioned into a text-only knowledge-agent platform. Each pivot left real artifacts behind.

PROBLEM

By this point the repository still carried: a full voice pipeline (Azure STT/TTS, an `/audio` route, browser mic recording) for an app that no longer used voice anywhere; dormant sales-stage logic (`conversation_stage.py`, `tools.py`) unreferenced by anything active; a local `documents/` folder superseded by Supabase Storage months earlier; unused frontend assets (default Vite boilerplate, leftover phone-call ambient audio files); stale `.gitignore` entries pointing at files and directories that no longer existed; and a README describing a completely different system — a different LLM provider, a different frontend stack, endpoints that no longer existed.

FIX

Multiple deliberate passes rather than one sweeping deletion: removed all `__pycache__` and build artifacts; deleted the dormant sales-stage files outright instead of leaving them dormant; deleted the superseded local storage folder; deleted every STT/TTS service file, the `/audio` routes, the related Azure/Google Cloud/pydub dependencies, and the frontend's audio API calls — confirmed with a whole-repository `grep` that returned nothing; rewrote both `.gitignore` files to match what actually exists; rewrote the README and a full architecture reference from scratch against the live, verified codebase rather than patching stale prose.

Every step in this chapter was verified the same way: compile check, boot check, and — where it touched a live code path — an actual request through the running backend, not just an assumption that removal was safe.

Closing a tab left the conversation "active" forever

Found by inspection, not by a test — the kind of gap that only shows up once you ask "what happens if the user just... leaves?"

PROBLEM

The Redis-backed session design (Chapter 07) made `/message` fast by keeping Postgres out of the hot path — but that meant a conversation's Postgres row only ever moved from `active` to `completed` when something explicitly called `/end`. If a user simply closed the tab, hit back, or navigated away mid-chat, nothing ever called it. The Postgres row sat at `status="active"` indefinitely — showing as a perpetually in-progress conversation on the organization's dashboard — while its Redis session quietly expired after 24 hours, taking the entire transcript with it, since messages are only ever persisted to Postgres at `/end`. Worse than a stale row: it was silent data loss with no record that anything had gone wrong.

FIX

Two layers, since neither one alone is reliable on its own:

- **Frontend — end-on-close beacon.** A `pagehide/beforeunload` listener in `Conversation.jsx` fires `navigator.sendBeacon()` at `/end` the moment the user leaves the page. A normal `fetch/XHR` call gets cancelled the instant the tab unloads; `sendBeacon` is the one request type browsers guarantee to still deliver. This handles the common case cleanly — full transcript persisted, summary generated, exactly as if "End Chat" was clicked.
- **Backend — opportunistic reconciliation.** For what even a beacon can miss (a crashed browser, no network at the moment of unload), any conversation still marked `active` whose Redis session has already vanished has no way left to ever reach `/end` on its own. A new `_reconcile_stale_active()` check runs whenever the organization dashboard lists or opens a conversation — no cron job or scheduler needed, since a dashboard view is itself a natural, frequent trigger — and flips any such conversation to `completed` on the spot.

Verified live with three conversations in parallel: one with its Redis session deleted (simulating expiry) correctly flipped to `completed` on the next dashboard read; a genuinely still-active one was left untouched in the same call; and a plain empty-body POST shaped exactly like what `sendBeacon` sends correctly finalized a conversation with its message history intact and a summary generated.

Known trade-off, stated rather than hidden: if a tab is abandoned and neither the beacon nor a dashboard visit happens before the 24-hour Redis TTL expires, the transcript itself is still lost — the conversation gets cleaned up to `completed` on the next reconciliation, but with no messages to persist, since Redis had already dropped them first. That's the direct cost of the Chapter 07 decision to keep Postgres off the hot path; shortening the reconciliation interval trades some of that risk back for more frequent Redis lookups.